

VDC Teacher Mobile Application

Detailed Implementation Plan

Project Overview

Project Name: VDC Teacher Mobile Application

Platform: Android APK

Frontend: React Native (Expo)

Backend: Supabase

Database: PostgreSQL

Authentication: Supabase Auth

Storage: Supabase Storage

SMS: Fast2SMS

Architecture Style

Cloud Native

Mobile First

Offline Ready

Production Ready

Phase 0 — Project Planning

Estimated Time

1 Day

Deliverables

- ✓ Product Requirements Document
- ✓ Technical Requirements Document
- ✓ Database Design
- ✓ UI Wireframes
- ✓ Navigation Flow
- ✓ User Stories

✓ API List

✓ Sprint Planning

Phase 1 — Environment Setup

Estimated Time

Half Day

Tasks

Create

GitHub Repository



Supabase Project



Development Branch



Production Branch



React Native Project



Folder Structure



Configure ESLint



Configure Prettier



Environment Variables



Install Dependencies

Deliverables

Running React Native project

Connected to Supabase

GitHub setup

Phase 2 — Database Development

Estimated Time

1 Day

Tasks

Design

Academic Tables



Authentication



Teacher Tables



Student Tables



Attendance



Test



Assignment



Notes



SMS



Reports



Indexes



Foreign Keys



Views



Triggers



RLS Policies

Deliverables

Production Database

Phase 3 — Authentication

Estimated Time

Half Day

Screens

Splash



Login



Forgot Password



Session Management



Logout

Features

JWT

Role Validation

Auto Login

Secure Storage

Deliverables

Teacher Login Complete

Phase 4 — UI Development

Estimated Time

2 Days

Develop

Theme



Typography



Components



Cards



Buttons



Dialogs



Bottom Navigation



Search



Forms



Dashboard



Profile



Settings

Deliverables

Complete UI Framework

Phase 5 — Dashboard

Estimated Time

Half Day

Widgets

Teacher Name

Today's Classes

Attendance Pending

Quick Actions

Reports

Recent Notes

Statistics

Deliverables

Functional Dashboard

Phase 6 — Class Module

Estimated Time

Half Day

Features

Assigned Classes



Semester



Section



Subject



Student Count



Class Details

Deliverables

Teacher can access classes

Phase 7 — Attendance Module

Estimated Time

1 Day

Workflow

Dashboard



Select Class



Student List



Present All



Mark Individual



Save



SMS



Success

Features

Search

Edit Attendance

Offline Cache

Sync

Deliverables

Complete Attendance Module

Phase 8 — SMS Integration

Estimated Time

Half Day

Workflow

Attendance Saved



Find Absent Students



Generate SMS



Call Fast2SMS



Store Logs



Success Screen

Deliverables

SMS Automation

Phase 9 — Test Module

Estimated Time

Half Day

Workflow

Create Test



Maximum Marks



Student List



Enter Marks



Validation



Save

Deliverables

Test Module

Phase 10 — Assignment Module

Estimated Time

Half Day

Workflow

Create Assignment



Student List



Enter Marks



Validation



Save

Deliverables

Assignment Module

Phase 11 — Notes Module

Estimated Time

Half Day

Workflow

Upload



Supabase Storage



Database



Preview



Download



Delete

Deliverables

Notes Repository

Phase 12 — Reports Module

Estimated Time

1 Day

Reports

Attendance

Monthly

Quarterly

Yearly

↓

Test

↓

Assignment

↓

Performance

↓

Charts

↓

PDF

↓

Excel

Deliverables

Complete Reports

Phase 13 — Offline Sync

Estimated Time

Half Day

Offline

Attendance

↓

Local Storage

↓

Internet Returns



Sync



Conflict Resolution

Deliverables

Offline Support

Phase 14 — Notifications

Version 1

Attendance Success

SMS Success

Errors

Future

Push Notifications

Phase 15 — Settings

Theme

Change Password

Logout

App Version

Privacy

About

Phase 16 — Testing

Estimated Time

2 Days

Testing

Unit Testing



Integration Testing



UI Testing



Regression Testing



Performance Testing



User Acceptance Testing



Bug Fixes

Deliverables

Release Candidate

Phase 17 — Deployment

APK



Teacher Testing



Feedback



Fixes



Production APK

Deliverables

Production Release

Folder Structure

src

assets

components

navigation

screens

hooks

services

api

store

utils

constants

types

theme

contexts

Development Order

Setup



Authentication



Database



Dashboard



Classes



Attendance



SMS



Tests



Assignments



Notes



Reports



Offline



Settings



Testing



Deployment

AI Development Workflow

Instead of asking AI to "build the app," break it into well-defined tasks.

Step 1

Generate database schema.



Step 2

Generate Supabase migrations.



Step 3

Generate React Native folder structure.



Step 4

Generate reusable UI components.



Step 5

Generate authentication flow.



Step 6

Generate attendance module.



Step 7

Generate test module.



Step 8

Generate assignment module.



Step 9

Generate reports.



Step 10

Generate notes.



Step 11

Testing.

This approach produces much cleaner and more maintainable code.

Git Branch Strategy

main

develop

feature/auth

feature/dashboard

feature/attendance

feature/tests

feature/assignments

feature/reports

feature/notes

feature/settings

release/v1.0

Milestones

Milestone Deliverable		Target
M1	Project setup complete	Day 1
M2	Database & Authentication	Day 2
M3	Dashboard & Class Management	Day 3
M4	Attendance + SMS	Day 4
M5	Tests + Assignments	Day 5
M6	Notes + Reports	Day 6
M7	Testing + Production APK	Day 7

Definition of Done

A feature is complete only if all of the following are true:

- Functional requirements implemented.
- UI matches the approved design.
- Input validation is complete.
- Error handling is implemented.
- Role-based access is enforced.
- Database transactions are safe.
- Responsive on supported Android screen sizes.

- Tested with success and failure scenarios.
- Reviewed and merged into the develop branch.
- Documented if new APIs or database changes were introduced.

My recommendation

Since you're using AI to accelerate development, I would split the project into **approximately 25–30 independent implementation tasks** (authentication, attendance, reports, notes, etc.) rather than 7 large phases. Each task should have:

- Objective
- Inputs
- Outputs
- Acceptance criteria
- Files to modify
- Test cases

This task-oriented approach aligns much better with Claude Code and Cursor, making the AI produce more accurate, reviewable, and maintainable code while reducing the chance of introducing regressions.